

Claude Code 기능 정리

Anthropic 공식 문서(code.claude.com/docs)를 기반으로 정리한 Claude Code의 전체 기능 지도 — 코어 구조부터 확장 레이어, 실행 플랫폼, 자동화까지.

작성일 2026년 8월 11일

출처 code.claude.com/docs · docs.claude.com (공식 문서 직접 조회)

대상 하네스 엔지니어링 학습·실무 참조용

목차

- 한눈에 보는 결론
- 코어 — 에이전틱 루프와 내장 도구
- 확장 레이어 8종
- 실행 플랫폼
- 세션 관리와 운영
- 자동화 · 코드 리뷰 · 보안
- Agent SDK
- 도입 권고 순서
- 출처

01 한눈에 보는 결론

Claude Code는 "터미널에서 도는 챗봇"이 아니라 **에이전틱 하네스(agentic harness)**입니다. 모델이 판단하고, 하네스가 도구·컨텍스트·권한·실행 환경을 붙여줍니다. 공식 문서의 표현 그대로, 하네스는 "언어 모델을 유능한 코딩 에이전트로 바꿔주는 층"입니다.

비유 — 모델이 "손이 좋은 요리사"라면 Claude Code는 **주방**입니다. 칼(내장 도구), 레시피북(CLAUDE.md·Skills), 식자재 배송망(MCP), 보조 요리사(Subagents), 타이머 알람(Hooks)을 갖춘 주방이요. 요리사만 바뀌서는 못 하는 일을 주방이 가능하게 만듭니다.

기능은 크게 네 덩어리로 나뉩니다.

- **코어** — 에이전틱 루프 + 5개 범주의 내장 도구
- **확장 레이어** — CLAUDE.md, Skills, Subagents, Agent teams, Code intelligence, MCP, Hooks, Plugins
- **실행 플랫폼** — 터미널, IDE, 데스크톱, 웹, 모바일, Chrome, CI
- **운영·자동화** — 권한 모드, 세션 관리, Routines, 코드 리뷰·보안

■ 그로 인해 생기는 상황 (파급)

- **설정을 안 하면 성능의 절반만 씁니다.** 확장 레이어는 옵션이 아니라 실질 성능 변수입니다.
- **반대로 처음부터 다 깔면 컨텍스트가 터집니다.** 공식 문서도 "트리거가 생길 때마다 하나씩 추가하라(build your setup over time)"고 명시합니다.
- **자동화 범위가 넓어질수록 권한 설계가 리스크 지점이 됩니다.** 파괴적 명령 차단은 하네스 쪽에서 잡아야 합니다.

02 코어 — 에이전틱 루프와 내장 도구

■ 에이전틱 루프

작업을 받으면 세 단계를 섞어가며 반복합니다. 단계마다 이전 결과를 보고 다음 행동을 정하고, 수십 개의 액션을 이어 붙이면서 스스로 궤도를 수정합니다.

① 컨텍스트 수집

파일 검색·읽기로 코드 이해

② 실행

파일 편집·명령 실행으로 변경

③ 결과 검증

테스트·빌드로 자기 작업 확인

루프는 요청에 따라 모양이 달라집니다. 단순 질문은 ①에서 끝나고, 버그 수정은 ①~③을 여러 번 돌며, 리팩토링은 검증에 비중이 실립니다. **사용자도 루프의 일부**여서 언제나 끼어들어 방향을 틀 수 있습니다.

모델

Sonnet은 대부분의 코딩 작업, Opus는 복잡한 아키텍처 판단에 강합니다. 세션 중에는 `/model` , 시작할 때는 `claude --model <name>` 으로 전환합니다.

내장 도구 5개 범주

범주	할 수 있는 일
파일 조작	파일 읽기, 코드 편집, 새 파일 생성
검색	파일명·내용 패턴 검색 (grep/glob 계열)
명령 실행	셸 명령, 빌드, 테스트, git 등 CLI 도구 전반
웹 접근	웹 검색 및 페이지 조회
외부 연동	MCP 서버를 통한 외부 서비스 호출

포인트 — 도구가 없으면 Claude는 텍스트로만 답합니다. 도구가 있어야 "읽고, 고치고, 돌려보고, 확인하는" 에이전트가 됩니다. 각 도구 호출 결과가 다시 루프로 들어가 다음 판단의 재료가 됩니다.

03 확장 레이어 8종

확장은 에이전틱 루프의 서로 다른 지점에 꽂힙니다. "매 세션 항상 보이는 컨텍스트" → "필요할 때 부르는 능력" → "이벤트에 자동으로 도는 백그라운드" 스펙트럼으로 이해하면 정리가 쉽습니다.

기능	하는 일	쓸 때
CLAUDE.md	매 대화에 자동 로드되는 영속 컨텍스트	프로젝트 컨벤션, "항상 X 해라" 규칙
Skills	지식·워크플로 묶음. <code>/deploy</code> 식 호출 또는 Claude가 필요할 때 자동 로드	반복 작업, 레퍼런스 문서, 재사용 콘텐츠
Subagents	격리된 컨텍스트에서 자체 루프를 돌고 <i>요약만</i> 반환	컨텍스트 절약, 병렬 작업, 전문 워커
Agent teams	독립 세션 여러 개를 태스크 공유 + P2P 메시징으로 조율	병렬 리서치, 신규 기능 개발, 경쟁 가설 디버깅
Code intelligence	언어 서버(LSP) 연결 — 심볼 단위 탐색, 실시간 타입 에러	타입 언어, grep이 느리거나 부정확한 대형 코드베이스
MCP	외부 서비스·도구 연결	DB 조회, Slack 전송, 브라우저 제어
Hooks	라이프사이클 이벤트에 스크립트·HTTP·프롬프트·서브에이전트 실행	"편집할 때마다 ESLint" 같은 <i>반드시</i> 돌아야 하는 자동화
Plugins	위 요소들을 하나의 설치 단위로 패키징 (+ Marketplaces로 배포)	여러 레포에서 재사용, 팀·외부 배포

특히 중요한 두 가지

① **Skills**가 가장 유연한 확장점입니다. 마크다운 파일 하나면 되고, 현재 대화에서 돌 수도 있고 서브에이전트로 격리해서 돌 수도 있습니다. 참고로 커스텀 커맨드는 **Skills**로 통합됐습니다 — `.claude/commands/deploy.md` 와 `.claude/skills/deploy/SKILL.md` 는 둘 다 `/deploy` 가 되고 동작이 같습니다.

② **Hooks**는 네 가지 타입을 지원합니다. 셸 명령, HTTP 엔드포인트, `type: "prompt"` (모델에 단발 평가 요청), `type: "agent"` (도구를 쓰는 서브에이전트 스폰), 그리고 `type: "mcp_tool"` (이미 연결된 MCP 서버의 도구 호출)까지 쓸 수 있습니다.

Artifacts — 여기에 더해, 세션 산출물을 *비공개 인터랙티브 웹 페이지*로 발행하는 기능이 있습니다. 터미널 스크롤백보다 시각적으로 보거나 공유해야 하는 결과물(조사 리포트, 타임라인, 대시보드)에 적합합니다.

Plugins는 "패키징 레이어"입니다. 플러그인 스킴은 `/my-plugin:review` 처럼 네임스페이스가 붙어서 여러 플러그인이 이름 충돌 없이 공존합니다.

04 실행 플랫폼

Claude Code는 터미널·IDE·데스크톱 앱·브라우저에서 돌고, 기존 워크플로를 바꾸지 않고 옆에 붙습니다.

환경	특징
터미널 CLI	기본 실행 환경. 네이티브 바이너리로 전환됨. Windows는 Git Bash 없이 동작
VS Code	인라인 diff, @-멘션, 플랜 리뷰, 단축키 지원
JetBrains	IntelliJ · PyCharm · WebStorm 등
데스크톱 앱	Git 격리 기반 병렬 세션, 드래그앤드롭 페인 배치, 내장 터미널·에디터·브라우저, 사이드 챗, 시각적 diff 리뷰, 앱 프리뷰, PR 모니터링, 예약 작업, iOS 시뮬레이터 페인. Linux 베타 / WSL 지원
웹 (claude.ai/code)	클라우드 실행. GitHub 레포 연결 → 작업 제출 → PR 리뷰. <code>--cloud</code> / <code>--teleport</code> 로 터미널 ↔ 웹 세션 이동
모바일	iOS·Android 앱에서 작업 시작·모니터링·조종. 푸시 알림
Remote Control	로컬 세션을 폰·태블릿·브라우저에서 그대로 이어받기
Chrome 연동	웹앱 테스트, 콘솔 로그 디버깅, 폼 자동 입력, 페이지 데이터 추출
Computer use	CLI에서 화면을 보고 앱 열기·클릭·타이핑 (macOS). 네이티브 앱 테스트, GUI 전용 도구 자동화
GitHub Actions / CI	<code>@claude</code> 멘션 대응, 이슈 → PR 자동화. Bedrock·Vertex 등 클라우드 제공자 연동

05 세션 관리와 운영

권한 모드

파일 편집·명령 실행 전에 물어볼지 여부를 제어합니다. CLI에서는 **Shift+Tab**으로 모드를 순환하고, VS Code·데스크톱·claude.ai에는 모드 선택기가 있습니다. 핸즈오프인 **auto mode**에서는 *hard deny* 규칙으로 특정 동작을 무조건 차단할 수 있고, *deny·ask* 규칙에 도구 파라미터 단위 매칭도 가능합니다.

세션

- `--continue` / `--resume` / `--from-pr` — 이어하기, 선택 재개, PR로부터 세션 복원
- `/resume` 피커에 PR URL을 붙여넣으면 그 PR을 만든 세션을 찾아줍니다
- `/rewind` — `/clear` 이전 대화로 되돌리기
- 세션 이름 지정, 브랜칭, 트랜스크립트 내보내기, **Ctrl+R**로 전 프로젝트 명령 히스토리 검색

■ 컨텍스트와 비용

- **프롬프트 캐싱 자동 관리** — 모델을 바꾸면 캐시가 깨져 한 번이 느려지고, 세션 중 CLAUDE.md 수정은 즉시 반영되지 않습니다
- `/compact` — 컨텍스트 압축 (비용이 발생함)
- `/usage` — 어떤 스킴·서브에이전트·MCP 서버가 플랜 한도를 소모하는지 분해해서 표시

■ 메모리

CLAUDE.md로 영속 지시를 주고, **auto memory**로 Claude가 스스로 학습 내용을 축적하게 할 수 있습니다.

■ 진단·설정

- `/config` — 프롬프트에서 모든 설정을 바로 변경
- `/doctor` — 셋업 전체 점검
- **safe mode** — 설정이 깨졌을 때 문제 해결용으로 부팅
- **agent view** — 모든 Claude Code 세션을 한 화면에서 관리

06 자동화 · 코드 리뷰 · 보안

자동화

기능	설명
Routines	클라우드 인프라에서 스케줄 / API 호출 / GitHub 이벤트로 자동 실행. Claude Code를 오토파일럿으로 돌리는 층
예약 작업 (Desktop)	데일리 코드 리뷰, 의존성 감사, 모닝 브리핑 같은 반복 작업을 데스크톱에서 스케줄링
<code>/loop</code> + Monitor	조건이 충족될 때까지 목표를 향해 계속 작업. 간격을 안 주면 스스로 페이스 조절
Dynamic workflows	큰 작업을 여러 단계로 오케스트레이션

코드 리뷰

- `/code-review` — diff 기반 리뷰
- `/code-review ultra` — 클라우드 멀티 에이전트 심층 리뷰. 버그를 찾고 검증까지 한 뒤 보고 (머지 전 사용)
- Code Review (GitHub)** — PR에 자동 리뷰. 로직 오류·보안 취약점·리그레션을 전체 코드베이스 맥락으로 탐지

보안

- security-guidance** 플러그인 — Claude가 자기가 쓴 코드를 같은 세션에서 취약점 관점으로 리뷰하고 수정
- Claude Security** 플러그인 — 코드베이스 전체 취약점 스캔 → 검토·적용 가능한 패치로 전환

07 Agent SDK

Claude Code의 기능(스킬·서브에이전트·훅·플러그인·슬래시 커맨드)을 직접 만드는 에이전트에 그대로 있는 SDK입니다. 로컬 디렉터리에서 플러그인을 프로그래밍적으로 로드해 커스텀 커맨드·에이전트·스킬·훅·MCP 서버를 세션에 주입할 수 있습니다.

서브에이전트를 정의하는 `--agents` 플래그는 다음 필드를 JSON으로 받습니다:

```
description · prompt · tools · disallowedTools · model · permissionMode ·  
mcpServers · hooks · maxTurns · skills · initialPrompt · memory · effort ·  
background · isolation · color
```

08 도입 권고 순서

공식 문서의 조언대로 **한 번에 다 세팅하지 않습니다**. 기능마다 "이럴 때 필요하다"는 알아보기 쉬운 트리거가 있고, 대부분의 팀은 그 트리거가 왔을 때 하나씩 붙입니다.

- 1 **CLAUDE.md** 먼저. 프로젝트 컨벤션과 "항상 이렇게 해라" 규칙을 여기에 고정합니다. 투자 대비 효과가 가장 큼니다.
- 2 **Skills**로 반복 작업 고정. 같은 지시를 세 번 이상 반복했다면 그건 스킬이 될 신호입니다.
- 3 **Hooks**는 "매번 반드시 돌아야 하는 것"만. lint, 포맷, 보안 가드 정도. 남발하면 매 턴 비용이 됩니다.
- 4 **MCP**는 외부 데이터·액션이 실제로 필요할 때. 연결만 해두면 컨텍스트만 먹습니다.
- 5 **Subagents / Agent teams**는 컨텍스트가 실제로 부족해질 때. 격리와 병렬성이 필요한 시점에 도입합니다.
- 6 **Plugins**는 마지막. 위 설정이 안정화되어 여러 레포·팀에 재사용할 단계가 됐을 때 패키징합니다.

정리 — 확장 레이어는 "많이 붙일수록 좋은 것"이 아니라 **트리거가 왔을 때 붙이는 것**입니다. 컨텍스트 윈도우는 유한하고, 안 쓰는 확장은 매 세션 세금처럼 자리를 차지합니다.

09 출처

아래 공식 문서를 2026년 8월 11일에 직접 조회하여 작성했습니다.

- Extend Claude Code (features overview) — code.claude.com/docs/en/features-overview
- How Claude Code works — code.claude.com/docs/en/how-claude-code-works
- Claude Code Docs 전체 인덱스 — code.claude.com/docs/llms.txt
- Overview — code.claude.com/docs/en/overview
- Hooks reference — docs.claude.com/en/docs/claude-code/hooks
- Create custom subagents — docs.claude.com/en/docs/claude-code/sub-agents
- Extend Claude with skills — docs.claude.com/en/docs/claude-code/slash-commands
- Agent SDK overview — docs.claude.com/en/docs/agent-sdk/overview

주의 — Claude Code는 주 단위로 기능이 추가됩니다. 최신 변경은 code.claude.com/docs/en/changelog 및 What's New 주간 문서를 확인하세요. 본 문서는 2026-08-11 기준 스냅샷입니다.